

TP Slurm 2/2

Nom et prénom :

Veillez recopier les commandes exécutées et les résultats de ces commandes (et/ou inclure des captures d'écran), EXPLIQUER LA COMMANDE et/ou LE RESULTAT, et répondre aux questions de ce document. Les réponses peuvent être rendues dans le format de votre choix (Word, PDF séparé, papier puis scanné), tant que les numérotations des questions/réponses sont respectées.

*Le document contenant les réponses est à envoyer à **lise.jolicoeur@cea.fr** en fin de TP **OU**, si non terminé, jusqu'au **23/02/2026** (inclus).*

Merci d'indiquer votre NOM dans le nom de fichier du document envoyé.

0. Préparation et validation de l'environnement

Dans cette introduction, nous allons démarrer une machine virtuelle pour préparer la configuration d'un cluster Slurm. Nous sauvegarderons ensuite l'image de cette VM puis l'instancierons dans 4 machines virtuelles pour disposer d'un cluster équivalent au cluster du TP slurm1. Nous aurons alors :

- **vm0** comme leader et worker slurm
- **vm[1-3]** comme worker slurm seulement

1. Démarrer **1** machine virtuelle avec PCOCC et le *template* du TP Slurm 2.

```
pcocc alloc -c2 mycentos74-tp-slurm2:1
```

2. Se connecter à cette machine virtuelle **vm0** :

```
pcocc ssh vm0
```

1. Mise en place et validation de munge

Dans un premier temps nous allons mettre en place munge afin de permettre la sécurisation des communications et des « job credentials » de Slurm.

Pour éviter d'avoir à taper sudo devant toutes les commandes, vous pouvez passer root, dans les VM, avec la commande 'sudo su'. Si vous décidez de faire ça, faites bien attention dans la suite du TP à quel utilisateur est censé réaliser la commande demandée.

1. Installer les packages munge :

```
sudo yum -y install munge
```

2. Créer le secret partagé de munge à l'aide de la commande du package :

```
sudo /usr/sbin/create-munge-key
```

3. Activer le service à chaque démarrage puis lancer le service :

```
sudo systemctl enable munge
```

```
sudo systemctl start munge
```

4. Tester le bon fonctionnement de munge. Commenter les résultats :

```
echo « Hello » | munge
```

```
echo « Hello » | munge | unmunge
```

5. Valider le fonctionnement de la détection du « replay ». Surligner/Mettre en avant l'élément qui vous confirme son fonctionnement. A partir des résultats, expliquer ce qu'est le 'replay' d'un message munge et pourquoi chaque message munge est protégé contre cela.

```
echo « Hello » | munge > /tmp/munge_cred
```

```
cat /tmp/munge_cred | unmunge
```

```
cat /tmp/munge_cred | unmunge
```

6. Valider le fonctionnement de l'expiration. Surligner l'élément qui vous confirme son fonctionnement. Expliquer pourquoi on impose l'expiration d'un message munge.

```
echo « Hello » | munge -t 30 > /tmp/munge_cred  
  
sleep 31; cat /tmp/munge_cred | unmunge
```

7. Valider le fonctionnement de la restriction des uid/gid autorisés à ouvrir les « munge credential ». Expliquer la différence entre les deux résultats. Expliquer pourquoi on souhaite restreindre la lecture des messages munge grâce aux uid/gid.

Attention, ces commandes ne doivent pas être exécutées en tant que root

```
echo « Hello » | munge -u 0 -g 0 | unmunge  
  
echo « Hello » | munge -u 0 -g 0 | sudo unmunge
```

2. Mise en place de mariadb et slurmdbd

Nous allons maintenant mettre en place un service MariaDB (MySQL) ainsi qu'un service Slurmdbd pour notre futur cluster Slurm.

1. Installer les packages nécessaires à MariaDB :

```
sudo yum -y install mariadb mariadb-server
```

2. Activer le service à chaque démarrage puis lancer le service :

```
sudo systemctl enable mariadb  
  
sudo systemctl start mariadb
```

- 3.

3.1 Accéder à MySQL, créer une base de données pour Slurm et autoriser le futur utilisateur de slurmdbd à y accéder. Analyser brièvement la deuxième commande effectuée grâce au man de mysql (<https://dev.mysql.com/doc/refman/8.0/en/grant.html>) pour comprendre comment se fait l'accès à la base de données.

```
sudo mysql  
  
create database slurm_acct_db ;  
  
grant all on slurm_acct_db.* TO 'root'@'localhost' identified by
```

```
'myslurmdbdpass' with grant option;  
  
flush privileges ;  
  
quit
```

3.2 Vérifier les restrictions d'accès et l'existence de la base de données pour Slurm.

```
sudo mysql  
  
sudo mysql -p  
  
show databases;  
  
quit
```

4. Installer les packages nécessaires à slurmdbd :

```
sudo yum -y install slurm-slurmdbd slurm-munge
```

5. Configurer slurmdbd en créant le fichier de configuration 'slurmdbd.conf' dans le dossier contenant la configuration de Slurm (/etc/slurm/slurmdbd.conf) et en le remplissant avec:

```
AuthType=auth/munge  
  
DbdHost=localhost  
  
SlurmUser=root  
  
DebugLevel=4  
  
LogFile=/var/log/slurm/slurmdbd.log  
  
PidFile=/var/run/slurmdbd.pid  
  
StorageType=accounting_storage/mysql  
  
StoragePass=myslurmdbdpass
```

```
StorageUser=root
```

Surligner les lignes de configuration qui vous paraissent pertinentes vis à vis de la configuration de MySQL précédente.

Lancer le service, puis consulter les logs généré pour slurmdbd et les commenter.

```
sudo mkdir /var/log/slurm  
  
sudo systemctl enable slurmdbd  
  
sudo systemctl start slurmdbd  
  
sudo cat /var/log/slurm/slurmdbd.log  
  
ps -elf|grep slurm
```

3. Mise en place et configuration de slurm

Nous allons maintenant mettre en place les composants principaux de Slurm puis les activer sur le cluster.

1. Installer les packages nécessaires à slurmctld/slurmd :

```
sudo yum -y install slurm slurm-munge
```

2. Configurer slurm en créant le fichier de configuration '/etc/slurm/slurm.conf' et en le remplissant avec les lignes données ci-dessous. Dans les lignes suivantes, surligner (ou colorer le texte) en :

- **vert** les lignes de configuration associées à slurmctld uniquement
- **jaune** les lignes de configuration associées à slurmd uniquement
- **bleu** les lignes de configuration associées aux deux composants

```
ClusterName=ensiie  
  
ControlMachine=vm0  
  
SlurmUser=root  
  
SlurmctldPort=6817
```

AuthType=auth/munge

StateSaveLocation=/tmp/slurmctld

SlurmdSpoolDir=/tmp/slurmd

SlurmctldPidFile=/var/run/slurmctld.pid

SlurmdPidFile=/var/run/slurmd.pid

SlurmctldTimeout=300

SlurmdTimeout=300

MessageTimeout=90

SelectType=select/cons_res

SelectTypeParameters=CR_Core

SchedulerType=sched/backfill

SlurmctldDebug=3

SlurmctldLogFile=/var/log/slurmctld.log

SlurmdDebug=3

SlurmdLogFile=/var/log/slurmd.log

AccountingStorageType=accounting_storage/slurmdbd

AccountingStorageHost=vm0

AccountingStorageEnforce=qos,limits

PriorityType=priority/multifactor

PriorityWeightFairShare=100000

PriorityWeightJobSize=0

PriorityWeightPartition=0

PriorityWeightQOS=1000000

```
NodeName=vm[0-3] Sockets=2 RealMemory=8000 State=UNKNOWN
```

```
PartitionName=vm Nodes=vm[0-3] Default=YES MaxTime=INFINITE  
State=UP
```

3. Il est maintenant nécessaire de créer le cluster dans slurmdbd et d'y générer les autres informations utiles. Dire quelle est la ou les lignes de configuration du fichier précédent (/etc/slurm/slurm.conf) qui nous y oblige.

Avec sacctmgr et l'option '-i', effectuer les actions suivantes en vous aidant du man de sacctmgr et en utilisant sur les notions de fairshare, d'association et de qos :

Exemple d'une commande sacctmgr pour ajouter un utilisateur : «sacctmgr -i add user name=«prenom.nom» cluster=ensiie account=guests fairshare=10 qos=normal,debug»

créer un cluster nommé 'ensiie'

créer un qos appelé 'debug', qui a une priorité de 10 et permet au maximum 1 job (priorité 'MaxJobs') de 5 minutes.

créer un compte (account) appelé 'guests' qui n'aura accès qu'à la moitié du cluster (50%)

ajouter votre utilisateur (récupérer son nom d'utilisateur avec la commande 'whoami') qui aura accès au cluster 'ensiie', à travers le compte 'guests', qui aura le droit d'utiliser 10 % du cluster (donc 1/5 des ressources de son compte) et qui pourra soumettre des jobs avec les qos normal et debug.

D'après le résultat de la commande 'sacctmgr show qos', quelle qos est la plus prioritaire ? Quel est le temps limite de ses jobs ?

4. Il est maintenant possible d'activer et de démarrer slurmctld sur la VM. Que constatez vous à l'issue du démarrage ?

```
sudo systemctl enable slurmctld
```

```
sudo systemctl start slurmctld
```

```
sinfo
```

5. Il est maintenant possible d'activer et de démarrer slurmd sur la VM. Décrire les changements que le démarrage de slurmd a engendré sur le cluster grâce à la sortie de sinfo.

```
sudo systemctl enable slurmd

sudo systemctl start slurmd

ps -elf|grep slurm

sinfo
```

6. Il est maintenant temps de **sauvegarder les modifications dans l'image PCOCC** de la VM afin de pouvoir arrêter le cluster et le lancer à nouveau avec 4 machines virtuelles :

Sortir de la VM actuelle pour revenir sur hpc01, par exemple avec la commande 'exit' (**une seule fois** ! Il ne faut pas sortir de l'allocation avant d'avoir sauvé la VM)

Sauver l'état de la VM avec la commande 'pcocc save'

Maintenant que les VM sont sauvées, on peut sortir de l'allocation et libérer les ressources, en faisant la commande 'exit'

Relance ensuite un cluster (par défaut, PCOCC lance la dernière version sauvegardée de l'image indiquée donc rien à modifier dans la commande):

```
pcocc alloc -c2 mycentos74-tp-slurm2:4
```

```
pcocc ssh vm0
```

Vous devriez maintenant disposer d'un cluster Slurm à peu près équivalent à celui utilisé pour le TP 'slurm1'. S'assurer que l'ensemble des nœuds du cluster soit dans l'état « idle » (cf TP1).

Dans l'éventualité d'un problème d'accès au service Slurm, il peut être nécessaire de redémarrer les services slurmdbd et slurmctld. Pour cela, faire sur vm0 :

```
sudo systemctl restart slurmdbd
```



```
sudo systemctl restart slurmctld  
  
sinfo  
  
si besoin mettre les nœuds idle avec scontrol
```

4. Mise en place d'une topologie réseau « Tree »

L'idée est maintenant de configurer Slurm de façon à lui indiquer que les 4 nœuds de calcul vm[0-3] sont répartis sur deux switches.

1. On commence par vérifier les nœuds actuellement alloués pour un job de deux nœuds :

```
srun -n 2 -N 2 hostname
```

2. Ajouter une ligne au fichier de configuration de slurm (/etc/slurm/slurm.conf) :

```
TopologyPlugin=topology/tree
```

Créer et remplir le fichier de configuration de la topologie de Slurm (/etc/slurm/topology.conf) avec les 3 lignes suivantes, puis redémarrer le service slurmctld sur vm0 pour prendre en compte la modification.

```
SwitchName=l1 Nodes=vm[0,2]  
  
SwitchName=l2 Nodes=vm[1,3]  
  
SwitchName=t0 Switches=l[1-2]
```

```
sudo systemctl restart slurmctld  
  
scontrol show topo
```

3. On peut maintenant relancer le job de la question 4.1. Constater et expliquer la différence de comportement. Expliquer les avantages liés à l'utilisation de la topologie.

```
srun -n 2 -N 2 hostname
```

5. Mise en place des cgroups pour le confinement et le binding des processus

L'idée est maintenant de configurer Slurm de façon à mettre en œuvre la logique des cgroup pour le confinement et le binding des processus utilisateurs.

4. On commence par vérifier les confinements et les binding initialement fournis par Slurm. Commenter les résultats obtenus.

```
srun -n 1 cat /proc/self/status | grep Cpus_allowed_list  
  
srun -n 2 cat /proc/self/status | grep Cpus_allowed_list
```

5. On ajoute ensuite la configuration nécessaire à la mise en place de cette topologie et on relance les services slurmctld et slurmd pour prise en compte :

Ajouter les lignes suivantes dans /etc/slurm/slurm.conf

```
TaskPlugin=task/cgroup  
TaskPluginParam=Cpusets,Cores
```

Créer et remplir le fichier /etc/slurm/cgroup.conf avec les lignes suivantes :

```
CgroupMountpoint=/dev/cgroup  
  
CgroupAutomount=yes  
  
ConstrainCores=yes  
  
TaskAffinity=yes
```

Redémarrer les services slurmctld et slurmd :

```
sudo systemctl restart slurmctld  
  
sudo systemctl restart slurmd
```

6. On peut maintenant relancer les commandes de la question 5.4. Constater et expliquer la différence de comportement.

```
srun -n 1 cat /proc/self/status | grep Cpus_allowed_list
```

```
srun -n 2 cat /proc/self/status | grep Cpus_allowed_list
```

7. Exécuter la commande suivante :

```
srun -n 8 -N 4 cat /proc/self/status | grep Cpus_allowed_list
```

Que remarquez-vous ? Expliquer pourquoi l'optimisation cgroup n'est pas appliquée partout.

8. (Optionnel) Sauver l'image PCOCC et lancer un nouveau cluster (procédure décrite dans la question 3.6), puis relancer la commande 5.7 pour vérifier que l'optimisation s'applique maintenant à l'ensemble du cluster.

6. Exécution d'une application MPI

Il est maintenant possible d'installer un environnement MPI afin de compiler et d'exécuter une application MPI via Slurm.

Pour cela, on commence par installer openmpi et gcc afin de disposer des dépendances nécessaires.

```
sudo yum install -y openmpi gcc
```

Il est alors possible d'écrire une application MPI, de la compiler, puis de l'exécuter au sein du cluster.

Créer un fichier *'hello.c'* dans votre home et le remplir (par exemple) avec le programme suivant en C :

```
#include <mpi.h>

#include <stdio.h>

int main(int argc, char** argv) {

    int comm_size;

    int comm_rank;

    char proc_name[MPI_MAX_PROCESSOR_NAME];

    int len;

    MPI_Init(NULL, NULL);
```

```
MPI_Comm_size(MPI_COMM_WORLD, &comm_size);

MPI_Comm_rank(MPI_COMM_WORLD, &comm_rank);

MPI_Get_processor_name(proc_name, &len);

printf("Hello from %s, rank [%d/%d]\n",
       proc_name, comm_rank, comm_size);

MPI_Finalize();

}
```

Ajouter OpenMPI au PATH courant puis compiler le programme :

```
export PATH=$PATH:/opt/openmpi/2.0.3/bin/

mpicc -o hello hello.c
```

Ceci fait, il est nécessaire de sauvegarder l'image PCOCC puis de relancer les VM afin de disposer des environnements MPI sur l'ensemble des VM (procédure décrite dans la question 3.6).

Maintenant, essayer de lancer le programme parallèle.

```
salloc -n 8 mpirun ./hello
```